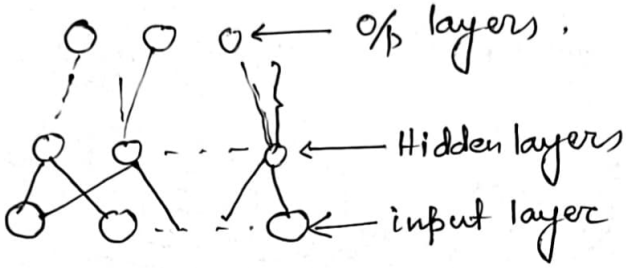
Derivation of gradient descent and the Delta rule in Multilayer Perceptron

Architecture -



1. **Set up and Notation**

   * Layers indexed by $l = 0, 1, \cdots L$,

   $l = 0$ — i/p layer..

   $l = L$ — o/p layer.

2. For neuron j in layer l:

   Pre-activation (input coming from the previous layer)

$$z_j^l = \sum_i w_{ji}\, a_i^{(l-1)} + b_j^{(l)}$$

   Activation -

$$a_j^l = \phi^l(z_j^{(l)})$$

   ϕ could be sigmoid, tanh or even it may be a linear function.

   Weight : $w_{ji}^{(l)}$ (from neuron i in previous layer).

   bias b_j^l :

in vector notation

$$z^{(l)} = w_{j1} a_1^{(l-1)} + w_{j2} a_2^{(l-1)} + \cdots + w_{jn} a_n^{(l-1)} + b_j^{(l)}$$

$$= W^{(l)} a^{(l-1)} + b^{(l)}.$$

* Loss (error) per training example, using mean square error —

$$E = \frac{1}{2} \sum_j (a_j^{(l)} - t_j)^2$$

* Our goal is to update weights using gradient descent.

$$w_{ji}^{(l)} \leftarrow w_{ji}^{(l)} - \eta \frac{\partial E}{\partial w_{ji}^{(l)}}$$ where η is the learning rate.

2. Delta rule for single neuron:

For one neuron —

$$z = \sum_i w_i x_i + b_\theta \leftarrow \text{pre activation.}$$

activation — $a = \phi(z)$

$$\text{Error} = \frac{1}{2}(a - t)^2$$

Gradient with respect to weight —

$$\frac{\partial E}{\partial w_i} = \frac{\partial E}{\partial a} \times \frac{\partial a}{\partial z} \times \frac{\partial z}{\partial w_i}$$

Compute terms

$$\frac{\partial E}{\partial a} = (a - t)$$

$$\frac{\partial a}{\partial z} = \phi'(z)$$

$$\frac{\partial z}{\partial w_i} = x_i$$

$$\therefore \quad \frac{\partial E}{\partial w_i} = (a - t)\,\phi'(z)\,x_i$$

Gradient descent update (the delta rule).

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i} = -\eta (a - t)\,\phi'(z)\,x_i$$

~~$= \Delta b\, x_i$~~

~~where~~ $\Delta b = -\eta (a - t)\, x_i$ *

If $\phi(z)$ is linear i.e. $\phi(z) = kz$ say

then $\phi'(z) = k$

$$\therefore \Delta w_i = -\eta \frac{\partial E}{\partial w_i} = -\eta (a - t)\, k x$$

$$= -\eta' (a - t)\, x \qquad \eta' = \eta k$$

This is called the Widrow-Hoff learning rule.

* $z = \sum w_i x_i + b$, $a = \phi(z)$ target t, $E = \frac{1}{2}(a-t)^2$

use chain rule –

$$\frac{\partial E}{\partial b} = \frac{\partial E}{\partial a} \times \frac{\partial a}{\partial z} \times \frac{\partial z}{\partial b}$$

$\frac{\partial E}{\partial a} = (a-t)$ $\frac{\partial a}{\partial z} = \phi'(z)$ $\frac{\partial z}{\partial b} = 1$

$$\therefore \frac{\partial E}{\partial b} = \phi'(z)(a-t)$$

$b \leftarrow b - \eta \frac{\partial E}{\partial b} \Rightarrow \Delta b = b_{new} - b_{old} = -\eta (a-t)\phi'(z)$

For $\phi(z)$ = sigmoidal or tanh one has to calculate $\phi'(z)$

The name of the gradient descent is delta rule.

The term "delta" refers to the error signal.

$$\delta = (t - a)(a - t)$$

which is literally the difference between the target output and the actual output.

In mathematics and engineering the greek letter δ (delta) is often used to denote a small change or difference.

Here the delta is the difference between what the neuron should have produced and what is actually produced.

So the update is called the "delta rule" because the weight change is proportional to this error difference (δ).

Multilayer Perceptron:

Define the error signal (delta) for neuron j in layer l:

$$\delta_j^{(l)} = \frac{\partial E}{\partial z_j^{(l)}}$$

Calculation of $\delta_j^{(l)}$ for output layers.

For output layer $\delta_j^{(l)} = \delta_j^{(L)}$

$$\delta_j^{(L)} = \frac{\partial E}{\partial z_j^{(L)}} = \frac{\partial E}{\partial a_j^{(L)}} \times \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}}$$

$$= \frac{\partial E}{\partial a_j^{(L)}} \times \phi'(z_j^{(L)})$$

$$= (a_j^{(L)} - t_j)\, \phi'(z_j^{(L)})$$

Next we calculate $\delta_j^{(l)}$ for hidden layers.

The pre activation of neuron k in layer $(l+1)$ is —

$$z_k^{(l+1)} = \sum_i w_{ki}^{(l+1)} a_i^{(l)} + b_k^{(l+1)}$$

Each $z_k^{(l+1)}$ depends on the activation $a_j^{(l)} = \phi^{(l)}(z_j^{(l)})$

Therefore by chain rule —

$$\delta_j^{(l)} = \frac{\partial E}{\partial z_j^{(l)}} = \sum \frac{\partial E}{\partial z_k^{(l+1)}} \times \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}}$$

We recognize $\frac{\partial E}{\partial z_k^{(l+1)}} = \delta_k^{(l+1)}$ so.

$$\delta_j^{(l)} = \sum_k \delta_k^{(l+1)} \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}}$$

Let us now compute $\frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}}$

observe $z_k^{l+1} = f(a^l)$

$a^l = \phi(z^l)$

∴ the chain dependence is —

$$z_k^{(l+1)} \longrightarrow a_j^{(l)} \longrightarrow z_j^{(l)}$$

$$\frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}} = \frac{\partial z_k^{(l+1)}}{\partial a_j^{(l)}} \times \frac{\partial a_j^{(l)}}{\partial z_j^{(l)}}$$

$$= w_{kj}^{(l+1)} \phi^{(l)'}(z_j^{(l)})$$

$$\therefore \delta_j^{(l)} = \sum_k \delta_k^{(l+1)} w_{kj}^{(l+1)} \phi^{(l)'}(z_j^{(l)})$$

$$= \phi^{(l)'}(z_j^{(l)}) \sum_k w_{kj}^{(l+1)} \delta_k^{(l+1)}$$

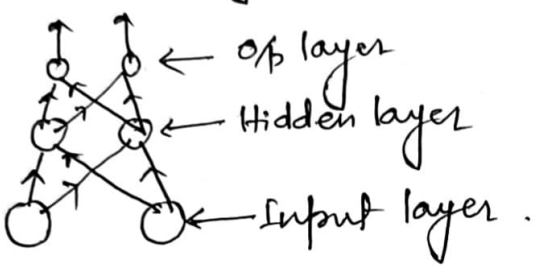
∴ update rules for multilayer perceptron is —

$$w_{ji}^{(l)} \leftarrow w_{ji}^{(l)} - \eta\, a_j^{(l-1)} \delta_j^{(l)}$$

$$b_j^{(l)} \leftarrow b_j^{(l)} - \eta\, \delta_j^{(l)}$$

Program 4 for Multilayer Perceptron

* Input layer: 2 ~~Next~~ Neurons.
* Hidden layer: 1 with 2 Neurons.
* Output layer: 2 Neuron.



$\mathbb{R}^2 \xrightarrow{\text{hidden(2)}} \xrightarrow{\text{output (2)}}$

Steps in Flow

Here is the logical flow broken into modules.

1. Start.
2. Initialize parameters.
   Randomly initialize weights $W^{(1)} \in \mathbb{R}^{2\times 2}$ bias $b^{(1)} \in \mathbb{R}^2$
   Randomly initialize weight $W^{(2)} \in \mathbb{R}^{2\times 2}$ bias $b^{(2)} \in \mathbb{R}^2$
   Choose learning rate η.

3. Input data:
   * Read training pair (x,t) where $x \in \mathbb{R}^2, t \in \mathbb{R}^2$

4. Forward Pass
   * Compute pre-activation –
     $z^{(1)} = W^{(1)}x + b^{(1)}$

* Hidden activation

$$a^{(1)} = \phi(z^{(1)})$$

* Output pre-activation —

$$z^{(2)} = W^{(2)} a^{(1)} + b^{(2)}$$

* Output activations:

$$a^{(2)} = \phi(z^{(2)})$$

5. Compute error

$$E = \frac{1}{2} \| a^{(2)} - t \|^2$$

6. Back pass (Back Propagation)

Output delta · $\delta^{(2)} = (a^{(2)} - t) \times \phi'(z^{(2)})$

Hidden layer $\delta^{(1)} = (W^{(2)^T} \delta^{(2)}) \times \phi'(z^{(1)})$

7. Parameter update —

update output weight & biases.

$$W^{(2)} \leftarrow W^{(2)} - \eta \delta^{(2)} (a^{(1)})^T$$

$$b^{(2)} \leftarrow b^{(2)} - \eta \delta^{(2)}$$

update hidden weights & biases:

$$W^{(1)} \leftarrow W^{(1)} - \eta \delta^{(1)} x^T$$

$$b^{(1)} \leftarrow b^{(1)} - \eta \delta^{(1)}$$

8. Repeat for all training samples (Epoch loop)

9. stop when error < threshold or more epochs reached.

10. End.

Flow Chart